

Per-turn integration pipeline

Replaces KELE's LLM-consultant + JSON-schema call with a CPU classifier + deterministic SocRule lookup

Algorithm 1 · Per-turn pipeline

Require: dialogue H , utterance u_t ,
classifier C , teacher T , map A , exemplars E

Ensure: response r_t , next state s_t

```
1:  s_t = C(format(H, u_t))
2:  if s_t.stage < current_stage(H):
3:      s_t = snap_forward(s_t)
4:  a_t = A[s_t]
5:  E_10 = stage_balanced_sample(E, k=10)
6:  r_t = T(H, u_t, a_t, E_10)
7:  return (r_t, s_t)
```

What the algorithm guarantees

Monotonic stage progression

lines 2–3

Socratic teaching can only move forward. `snap_forward` enforces this even if the classifier predicts an earlier stage.

Stage-balanced exemplar sampling

line 5

2 exemplars per SocRule stage. Without balancing, stage c (33.7% of the corpus) would dominate the teacher's example pool.

Explicit state-machine threading

line 7

Each turn returns s_t for the next turn's monotonicity check. Dialogue is a debuggable state machine, not an LLM-implicit buffer.

End-to-end determinism

lines 1, 4, 5

Classifier + state-action lookup + fixed-seed sampling. Same input always produces the same output. Reproducible and audit-able.

Takeaway: each numbered step encodes a structural guarantee that an LLM-consultant pipeline cannot provide